

TEST

1. ¿Qué ocurre cuando hacemos Cuenta c2 = c1;?
a) Se copia el objeto completo
2. ¿Cuál es correcta respecto a this?
c) Hace referencia al objeto actual en ejecución
3. Si un atributo es static:
d) Solo puede ser private
4. Si una excepción no es capturada:
d) Se convierte en RuntimeException
5. ¿Qué ocurre al crear: Cuenta[] cuentas = new Cuenta[3];?
b) Se crean 3 referencias a null

EXPLICA CÓDIGO

```
int[] a = {1,2,3};  
int[] b = a;  
int[] c = a.clone();  
  
b[0] = 50;  
c[1] = 100;  
  
System.out.println(a[0] + " " + a[1]);
```

Se crea un array de tipo int llamado a que contiene 3 elementos

Se crea otro array "b" tipo int que copia al array "a" y dichos valores tambien se verán afectados si se modifican en uno u otro

Se crea otro array "c" tipo int que clona la estancia del array a en este momento

El elemento con posición 0 del array "b" (sería el elemento con valor 1 dentro del array), cambia de valor a "50" y el elemento con posicion 1 del array "c" (sería el elemento con

valor 2) cambia a valor "100"

Imprime por pantalla el elemento posicion 0 del array a y seguidamente el elemento posicion 1 del mismo array

Tener en cuenta que el valor del elemento "0" fue modificado para el array "b" por lo que también para el array "a"

Tras explicar el código responde:

- ¿Qué se imprime por pantalla?

Imprimirá 50 2

- ¿Y si se hace un print de b[3] qué ocurre?

No existe la posicion 3 dentro del array "b" ya que sí existen 3 elementos pero su posicion va por : 0, 1, 2

ERRORES

```
public class Cuenta {  
  
    private double saldo;  
  
    public Cuenta(double saldo){  
        if(saldo >= 0)  
            this.saldo = saldo;  
    }  
  
    public void retirar(double cantidad){  
        if(cantidad > saldo)  
            throw new Exception("Saldo insuficiente");  
        saldo -= cantidad;  
    }  
}
```

- ¿Qué errores tiene?

Le falta un else en el método retirar tras la condicion del if y lanzar un Exception, porque lo que indica el codigo es que si la cantidad a retirar es mayor que la del saldo, te lance una excepción con un mensaje de saldo insuficiente pero a su misma vez también quita del saldo la cantidad introducida, el funcionamiento está mal

- Corrige correctamente usando excepciones

```
public void retirar(double cantidad) {  
    if (cantidad > saldo)  
        throw new Exception("Saldo insuficiente"); // Sigue llamando al constructor de Exception  
    } else { // Aqui se define mejor el funcionamiento dividiendo el camino que realiza el codigo  
        saldo -= cantidad;  
    }  
}
```

PREGUNTAS CORTAS

- Explica por qué los atributos deben ser privados.

Porque es una información que no se debe visualizar desde fuera, se debe encapsular obligatoriamente

- Explica qué es una clase y qué es un objeto. Crea una clase sencilla y un objeto a partir de dicha clase.

Una clase digamos que es el cajon o espacio donde se introducen el resto de codigo como metodos, objetos y es la clase la que define cómo se comporta y agrupa todo lo necesario para su funcionamiento

Un objeto es una entidad "única" que contiene unos atributos propios y puede usar los metodos que existan dentro de la misma clase

```
public class ejemplo() {  
    public static void main (String[] args) {  
        String coche = new Coche(); // Creo una instancia nueva del objeto coche  
        llamando al constructor  
    }  
}
```

```
public static class Coche() { // Creo la clase Coche  
    private String matricula;  
  
}
```

```
public Coche (String matricula) { // Constructor de la clase coche
    this.matricula = matricula;
}
public String getMatricula() { // Metodo un get para el atributo matricula
    return matricula;
}

public String setMatricula() { // Metodo el set para el atributo matricula
    this.matricula = matricula;
    return matricula;
}
```